



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7
по курсу «Языки и методы программирования»
«Разработка простейшего класса на C++»

Студент группы ИУ9-22Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

Выполнение лабораторной работы заключается в составлении на языке C++ программы, состоящей из трёх файлов:

- заголовочный файл `declaration.h` с объявлением заданных классов;
- файл `implementation.cpp` с определениями методов класса;
- файл `main.cpp`, содержащий функцию `main` и, возможно, вспомогательные функции для проверки работоспособности класса.

Реализация класса не должна опираться на стандартные контейнерные классы C++, то есть внутреннее состояние объектов класса должно быть реализовано через обычные массивы. Соответственно, в классе обязательно требуется реализовать:

- конструктор копий;
- деструктор (должен быть объявлен виртуальным);
- операцию присваивания.

Проверку работоспособности класса требуется организовать в функции `main`, размещённой в файле `main.cpp`.

Проверка должна включать в себя:

- создание объекта класса в автоматической памяти;
- передачу объекта класса по значению в функцию;
- присваивание объекта класса переменной.

1. Квадратная матрица, элементы которой являются рациональными числами, с операциями:

1. получение порядка матрицы;
2. получение ссылки на указанный элемент;
3. формирование подматрицы, полученной путём вычёркивания i -той строки и j -го столбца;
4. вычисление определителя матрицы.

Для представления рациональных чисел требуется реализовать класс нормализованных дробей с необходимыми арифметическими операциями. Конструктор матрицы должен принимать в качестве параметра её порядок и формировать нулевую матрицу.

2. Последовательность символов ASCII с операциями:
 1. получение количества символов;
 2. получение ссылки на i -тый символ;
 3. вставка нового символа в i -тую позицию последовательности;
 4. проверка, является ли последовательность палиндромом.

2 Результаты

Исходный код программы представлен в листингах 1–6.

Листинг 1 — Реализация файла main.cpp первой задачи

```
1 #include "declaration.h"
2 #include <iostream>
3 using namespace std;
4 int main() {
5     RationalMatrix mat(3);
6     mat.getElement(0, 0) = Rational(1, 2);
7     mat.getElement(0, 1) = Rational(3, 4);
8     mat.getElement(0, 2) = Rational(5, 6);
9     mat.getElement(1, 0) = Rational(7, 8);
10    mat.getElement(1, 1) = Rational(9, 10);
11    mat.getElement(1, 2) = Rational(11, 12);
12    mat.getElement(2, 0) = Rational(13, 14);
13    mat.getElement(2, 1) = Rational(15, 16);
14    mat.getElement(2, 2) = Rational(17, 18);
15    cout << "The initial matrix:" << std::endl;
16    for (int i = 0; i < mat.getOrder(); ++i) {
17        for (int j = 0; j < mat.getOrder(); ++j) {
18            cout << mat.getElement(i, j) << " ";
19        }
20        cout << std::endl;
21    }
22    cout << "The order of the matrix: " << mat.getOrder() << std::endl;
23    cout << "Element in (1, 1): " << mat.getElement(1, 1) << std::endl;
24    RationalMatrix submat = mat.getSubmatrix(1, 1);
25    cout << "The submatrix after conversion:" << std::endl;
26    for (int i = 0; i < submat.getOrder(); ++i) {
27        for (int j = 0; j < submat.getOrder(); ++j) {
28            cout << submat.getElement(i, j) << " ";
29        }
30        cout << std::endl;
31    }
32    cout << "The determinant of the matrix: " << mat.determinant() <<
std::endl;
33    return 0;
34 }
```

Листинг 2 — Реализация файла declaration.h первой задачи

```
1 #ifndef RATIONAL_MATRIX_H
2 #define RATIONAL_MATRIX_H
3 #include <iostream>
4 class Rational {
5 private:
6     int numerator;
7     int denominator;
8 public:
9     Rational(int num = 0, int denom = 1);
10    Rational(const Rational& other);
11    int abs(int n);
12    ~Rational();
13    Rational& operator=(const Rational& other);
14    void normalize();
15    int findGCD(int a, int b);
16    Rational operator+(const Rational& other) const;
17    Rational operator-(const Rational& other) const;
18    Rational operator*(const Rational& other) const;
19    Rational operator/(const Rational& other) const;
20    friend std::ostream& operator<<(std::ostream& os, const Rational&
    rat);
21 };
22 class RationalMatrix {
23 private:
24     int order;
25     Rational** data;
26 public:
27     RationalMatrix(int n);
28     RationalMatrix(const RationalMatrix& other);
29     ~RationalMatrix();
30     RationalMatrix& operator=(const RationalMatrix& other);
31     int getOrder() const;
32     Rational& getElement(int row, int col);
33     RationalMatrix getSubmatrix(int row, int col) const;
34     Rational determinant() const;
35 };
36
37 #endif // RATIONAL_MATRIX_H
```

Листинг 3: Реализация файла implementation.cpp первой задачи

```
1 #include "declaration.h"
2 #include <iostream>
3 using namespace std;
4 Rational::Rational(int num, int denom) : numerator(num), denominator(
    denom) {
5     normalize();
6 }
7 Rational::Rational(const Rational& other) : numerator(other.numerator),
    denominator(other.denominator) {}
8 Rational::~~Rational() {}
9 Rational& Rational::operator=(const Rational& other) {
10     if (this != &other) {
11         numerator = other.numerator;
12         denominator = other.denominator;
13     }
14     return *this;
15 }
16 int Rational::abs(int n) {
17     return n < 0 ? -n : n;
18 }
19 void Rational::normalize() {
20     if (denominator < 0) {
21         numerator *= -1;
22         denominator *= -1;
23     }
24     int gcd = findGCD(abs(numerator), denominator);
25     numerator /= gcd;
26     denominator /= gcd;
27 }
28 int Rational::findGCD(int a, int b) {
29     while (b != 0) {
30         int temp = b;
31         b = a % b;
32         a = temp;
33     }
34     return a;
35 }
36 Rational Rational::operator+(const Rational& other) const {
37     return Rational(numerator * other.denominator + other.numerator *
        denominator, denominator * other.denominator);
38 }
39 Rational Rational::operator-(const Rational& other) const {
40     return Rational(numerator * other.denominator - other.numerator *
        denominator, denominator * other.denominator);
41 }
```

```

42 Rational Rational::operator*(const Rational& other) const {
43     return Rational(numerator * other.numerator, denominator * other.
denominator);
44 }
45 Rational Rational::operator/(const Rational& other) const {
46     return Rational(numerator * other.denominator, denominator * other.
numerator);
47 }
48 std::ostream& operator<<(std::ostream& os, const Rational& rat) {
49     os << rat.numerator;
50     if (rat.denominator != 1) {
51         os << "/" << rat.denominator;
52     }
53     return os;
54 }
55 RationalMatrix::RationalMatrix(int n) : order(n) {
56     data = new Rational*[n];
57     for (int i = 0; i < n; ++i) {
58         data[i] = new Rational[n];
59     }
60 }
61 RationalMatrix::RationalMatrix(const RationalMatrix& other) : order(
other.order) {
62     data = new Rational*[order];
63     for (int i = 0; i < order; ++i) {
64         data[i] = new Rational[order];
65         for (int j = 0; j < order; ++j) {
66             data[i][j] = other.data[i][j];
67         }
68     }
69 }
70 RationalMatrix::~~RationalMatrix() {
71     for (int i = 0; i < order; ++i) {
72         delete [] data[i];
73     }
74     delete [] data;
75 }
76 RationalMatrix& RationalMatrix::operator=(const RationalMatrix& other) {
77     if (this != &other) {
78         for (int i = 0; i < order; ++i) {
79             delete [] data[i];
80         }
81         delete [] data;
82
83         order = other.order;
84         data = new Rational*[order];

```

```

85         for (int i = 0; i < order; ++i) {
86             data[i] = new Rational[order];
87             for (int j = 0; j < order; ++j) {
88                 data[i][j] = other.data[i][j];
89             }
90         }
91     }
92     return *this;
93 }
94 int RationalMatrix::getOrder() const { return order; }
95 Rational& RationalMatrix::getElement(int row, int col) {
96     return data[row][col];
97 }
98
99 RationalMatrix RationalMatrix::getSubmatrix(int row, int col) const {
100     RationalMatrix submatrix(order - 1);
101     for (int i = 0, k = 0; i < order; ++i) {
102         if (i == row) continue;
103         for (int j = 0, l = 0; j < order; ++j) {
104             if (j == col) continue;
105             submatrix.getElement(k, l) = data[i][j];
106             ++l;
107         }
108         ++k;
109     }
110     return submatrix;
111 }
112 Rational RationalMatrix::determinant() const {
113     if (order == 1) {
114         return data[0][0];
115     }
116     Rational det;
117     for (int j = 0; j < order; ++j) {
118         RationalMatrix submatrix = getSubmatrix(0, j);
119         det = det + (data[0][j] * (j % 2 == 0 ? 1 : -1) * submatrix.
120 determinant());
121     }
122     return det;
123 }

```

Результат запуска первой задачи представлен на рисунке 1.

```
Первоначальная матрица:
1/2 3/4 5/6
7/8 9/10 11/12
13/14 15/16 17/18
Порядок матрицы: 3
Элемент в (1, 1): 9/10
Подматрица после преобразования:
1/2 5/6
13/14 17/18
Определитель матрицы: 29/26880
...Program finished with exit code 0
```

Рис. 1 — Результат

Листинг 4 — Реализация файла main.cpp второй задачи

```
1 #include "declaration.h"
2 #include <cstring>
3 #include <iostream>
4 using namespace std;
5 int main() {
6     CharacterSequence seq;
7     seq.insert(0, 'a');
8     seq.insert(1, 'b');
9     seq.insert(2, 'c');
10    seq.insert(2, 'b');
11    seq.insert(3, 'a');
12    seq.insert(0, 'c');
13    std::cout << "Sequence: ";
14    for (int i = 0; i < seq.getLength(); ++i) {
15        std::cout << seq[i];
16    }
17    std::cout << std::endl;
18    std::cout << "The number of characters in the sequence: " << seq.
getLength() << std::endl;
19    std::cout << "i-th character: " << seq[2] << std::endl;
20    if (seq.isPalindrome()) {
21        std::cout << "This is a palindrome" << std::endl;
22    } else {
23        std::cout << "No, this is not a palindrome" << std::endl;
24    }
25    return 0;
26 }
```

Листинг 5: Реализация файла declaration.h второй задачи

```
1 #ifndef CHARACTER_SEQUENCE_H
2 #define CHARACTER_SEQUENCE_H
3 class CharacterSequence {
4 private:
5     char* data;
6     int length;
```



```

7 public :
8     CharacterSequence(const char* str = "");
9     CharacterSequence(const CharacterSequence& other);
10    ~CharacterSequence();
11    CharacterSequence& operator=(const CharacterSequence& other);
12    int getLength() const;
13    char& operator [] (int index);
14    const char& operator [] (int index) const;
15    void insert(int index, char ch);
16    bool isPalindrome() const;
17 };
18 #endif // CHARACTER_SEQUENCE_H

```

Листинг 6: Реализация файла implementation.cpp второй задачи

```

1 #include "declaration.h"
2 #include <cstring>
3 #include <iostream>
4 CharacterSequence::CharacterSequence(const char* str) : length(strlen(
    str)) {
5     data = new char[length + 1];
6     strcpy(data, str);
7 }
8 CharacterSequence::CharacterSequence(const CharacterSequence& other) :
    length(other.length) {
9     data = new char[length + 1];
10    strcpy(data, other.data);
11 }
12 CharacterSequence::~~CharacterSequence() {
13     delete [] data;
14 }
15 CharacterSequence& CharacterSequence::operator=(const CharacterSequence&
    other) {
16     if (this != &other) {
17         delete [] data;
18         length = other.length;
19         data = new char[length + 1];
20         strcpy(data, other.data);
21     }
22     return *this;
23 }
24 int CharacterSequence::getLength() const {
25     return length;
26 }
27 char& CharacterSequence::operator [] (int index) {
28     return data[index];
29 }

```

```

30 const char& CharacterSequence::operator [](int index) const {
31     return data[index];
32 }
33 void CharacterSequence::insert(int index, char ch) {
34     if (index >= 0 && index <= length) {
35         char* newData = new char[length + 2];
36         strncpy(newData, data, index);
37         newData[index] = ch;
38         strncpy(newData + index + 1, data + index, length - index + 1);
39         delete[] data;
40         data = newData;
41         length++;
42     }
43 }
44 bool CharacterSequence::isPalindrome() const {
45     for (int i = 0; i < length / 2; ++i) {
46         if (data[i] != data[length - i - 1]) {
47             return false;
48         }
49     }
50     return true;
51 }

```

Результат запуска второй задачи представлен на рисунке 2.

```

Последовательность: cabbac
Количество символов в последовательности: 6
i-тый символ: b
Это палиндром

...Program finished with exit code 0
Press ENTER to exit console.

```

Рис. 2 — Результат